Practical 1(a): Create an MVC Application to Perform Basic Arithmetic Operations

Models/Calculator.cs

```
namespace ArithmeticMVC.Models
{
    public class Calculator
    {
        public double Number1 { get; set; }
        public double Number2 { get; set; }
        public required string Operation { get; set; }
        public double Result { get; set; }
    }
}
```

Controllers/HomeController.cs

```
Using Microsoft.AspNetCore.Mvc;
Using ArithmeticMVC.Models;

namespace ArithmeticMVC.Controllers
{
    public class HomeController:Controller
    {
        // GET Method
        public IActionResult Index()
        {
            return View();
        }

        // POST Method
        [HttpPost]
        public IActionResult Index(Calculator calc)
        {
            switch (calc.Operation)
            {
                case "+":
                    calc.Result = calc.Number1 + calc.Number2;
                    break;
                case "-":
                    calc.Result = calc.Number1 - calc.Number2;
                    break;
                case "*":
                    calc.Result = calc.Number1 * calc.Number2;
                    break;
                case "/":
                    if (calc.Number2 != 0)
                    {
                        calc.Result = calc.Number1 / calc.Number2;
                    }
                    break;
            }
            return View(calc);
        }
    }
}
```

Views/Home/Index.cshtml

```
@model ArithmeticMVC.Models.Calculator

<h2>Arithmetic Operations</h2>
<form method="post">
<label>Enter Number1</label><br/>
```

```
<input type="number" step="any"asp-for="Number1"/>
<br/><br/>
<label>Enter Number2</label><br/>
<input type="number" step="any"asp-for="Number2"/>
<br/><br/>
<label>Select Operation</label><br/>
<select asp-for="Operation">
<option value="+">Addition</option>
<option value="-">Subtraction</option>
<option value="*">Multiplication</option>
<option value="/">Division</option>
</select>
<br/><br/>
<input type="submit" value="Calculate"/>
</form>
<br/>
@if(Model!=null)
{
<h3>Result=@Model.Result</h3>
}
```

Practical 1(b): Floyd's Triangle till N Rows in C#

```
using System;
class Program
{
    static void Main()
    {
        int n, number = 1;
        Console.Write("Enter number of rows:");
        n = Convert.ToInt32(Console.ReadLine());
        for(int i =1;i <=n;i++)
        {
            for(int j=1;j <=i;j++)
            {
                Console.Write(number + " ");
                number++;
            }
            Console.WriteLine();
        }
        Console.ReadLine();
    }
}
```

Practical 1(c): Arrange Names of Cities in Sorted Order using C#

```
using System;
class Program
{
    static void Main()
    {
        string[] cities = new string[10];
        Console.WriteLine("Enter 10 city names:");
        for(int i=0;i<10;i++)
        {
            cities[i] = Console.ReadLine();
        }
        Array.Sort(cities);
        Console.WriteLine("\nCities in Sorted Order:");
        foreach (string city in cities)
        {
            Console.WriteLine(city);
        }
        Console.ReadLine();
    }
}
```

Practical 2(a): Function Overloading in C#

```
using System;
class Addition
{
    //Function with 2 parameters
    public void Add(int a, int b)
    {
        Console.WriteLine("Addition of 2 numbers=" + (a + b));
    }
    //Overloaded function with 3 Parameters
    public void Add(int a, int b, int c)
    {
        Console.WriteLine("Addition of 3 numbers=" + (a + b + c));
    }
}
class Program
{
    static void Main()
    {
        Addition obj = new Addition();
        obj.Add(10, 20);
        obj.Add(10, 20, 30);
        Console.ReadLine();
    }
}
```

Practical 2(b): Inheritance (All Types) in C#

```
using System;
// Single Inheritance
class Animal
{
    public void Eat()
    {
        Console.WriteLine("Animal can Eat");
    }
}
class Dog : Animal
{
    public void Bark()
    {
        Console.WriteLine("Dog can Bark");
    }
}
// Multilevel Inheritance
class Puppy : Dog
{
    public void Weep()
    {
        Console.WriteLine("Puppy can Weep");
    }
}
// Hierarchical Inheritance
class Cat : Animal
{
    public void Meow()
    {
        Console.WriteLine("Cat can Meow");
    }
}
class Program
{
    static void Main()
    {
        // Single Inheritance
        Dog d = new Dog();
        d.Eat();
        d.Bark();
        // Multilevel Inheritance
        Puppy p = new Puppy();
        p.Eat();
        p.Bark();
        p.Weep();
        // Hierarchical Inheritance
        Cat c = new Cat();
        c.Eat();
        c.Meow();
        Console.ReadLine();
    }
}
```

Practical 2(c): Constructor Overloading in C#

```
using System;
class Student
{
    //Default Constructor
    public Student()
    {
        Console.WriteLine("Default Constructor Called");
    }
    //Parameterized Constructor
    public Student(string name)
    {
        Console.WriteLine("Student Name:" + name);
    }
    //Another Parameterized Constructor
    public Student(string name, int age)
    {
        Console.WriteLine("Student Name:" + name);
        Console.WriteLine("Student Age:" + age);
    }
}
class Program
{
    static void Main()
    {
        Student s1 = new Student();
        Student s2 = new Student("Rahul");
        Student s3 = new Student("Amit", 20);
        Console.ReadLine();
    }
}
```

Practical 2(d): Interfaces in C#

```
using System;
// Interface
interface IShape
{
    void Area();
}
// Class implementing interface
class Square : IShape
{
    int side = 5;
    public void Area()
    {
        Console.WriteLine("Area of Square = " + (side* side));
    }
}
class Program
{
    static void Main()
    {
        Square s = new Square();
        s.Area();
        Console.ReadLine();
    }
}
```

Practical 2(e): Delegates and Events in C#

```
using System;
// Delegate Declaration
public delegate void Message();
class Test
{
    // Event Declaration
    public event Message Notify;
    public void Show()
    {
        Console.WriteLine("Method inside Test class");
        // Raising Event
        Notify();
    }
}
class Program
{
    // Event Handling Method
    public static void Display()
    {
        Console.WriteLine("Event Executed Successfully");
    }
    static void Main()
    {
        Test t = new Test();
        // Attaching method to event
        t.Notify += Display;
        t.Show();
        Console.ReadLine();
    }
}
```

Practical 2(f): Exception Handling in C#

```
using System;
class Program
{
    static void Main()
    {
        int a, b, result;
        try
        {
            Console.Write("Enter First Number: ");
            a = Convert.ToInt32(Console.ReadLine());
            Console.Write("Enter Second Number: ");
            b = Convert.ToInt32(Console.ReadLine());
            result = a / b;
            Console.WriteLine("Result = " + result);
        }
        catch (DivideByZeroException)
        {
            Console.WriteLine("Cannot divide by zero");
        }
        catch (FormatException)
        {
            Console.WriteLine("Invalid Input");
        }
        finally
        {
            Console.WriteLine("Program Ended");
        }
        Console.ReadLine();
    }
}
```

Practical 3(a): Methods using C# Delegates

```
using System;
// Delegate Declaration
public delegate void Calculation(int a, int b);
class Arithmetic
{
    public void Add(int a, int b)
    {
        Console.WriteLine("Addition = " + (a + b));
    }
    public void Subtract(int a, int b)
    {
        Console.WriteLine("Subtraction = " + (a - b));
    }
    public void Multiply(int a, int b)
    {
        Console.WriteLine("Multiplication = " + (a* b));
    }
    public void Divide(int a, int b)
    {
        Console.WriteLine("Division = " + (a / b));
    }
}
class Program
{
    static void Main()
    {
        Arithmetic obj = new Arithmetic();
        Calculation c;
        c = obj.Add;
        c(20, 10);
        c = obj.Subtract;
        c(20, 10);
        c = obj.Multiply;
        c(20, 10);
        c = obj.Divide;
        c(20, 10);
        Console.ReadLine();
    }
}
```

Practical 3(b): Multicast Delegate

```
using System;
// Delegate Declaration
public delegate void Demo();
class Test
{
    public void Show()
    {
        Console.WriteLine("Show Method Called");
    }
    public void Display()
    {
        Console.WriteLine("Display Method Called");
    }
    public void Print()
    {
        Console.WriteLine("Print Method Called");
    }
}
class Program
{
    static void Main()
    {
        Test t = new Test();
        // Creating Multicast Delegate
        Demo d;
        d = t.Show;
        d += t.Display;
        d += t.Print;
        // Calling all methods together
        d();
        Console.ReadLine();
    }
}
```

Practical 3(c): BankAccount Class with Property Validation

```
using System;
class BankAccount
{
    // Field
    private double balance;
    // Property
    public double Balance
    {
        get
        {
            return balance;
        }
        set
        {
            if (value >= 0)
            {
                balance = value;
            }
            else
            {
                Console.WriteLine("Balance cannot be negative");
            }
        }
    }
    // Property for Account Number
    public int AccountNumber { get; set; }
}
class Program
{
    static void Main()
    {
        BankAccount b = new BankAccount();
        b.AccountNumber = 101;
        b.Balance = 5000;
        Console.WriteLine("Account Number: " + b.AccountNumber);
        Console.WriteLine("Balance: " + b.Balance);
        // Negative Balance Test
        b.Balance = -2000;
        Console.ReadLine();
    }
}
```

Practical 4(a): ASP.NET Web Forms: Student Registration Form

```
Protected Sub btnSubmit_Click(sender As Object, e As EventArgs) Handles btnSubmit.Click
    Dim gender As String = ""
    If RbMale.Checked Then
        gender = "Male"
    ElseIf rbFemale.Checked Then
        gender = "Female"
    End If
    If chkTerms.Checked Then
        lblOutput.Text = "Student Name : " & txtName.Text &
        "<br/>Gender : " & gender &
        "<br/>Course : " & ddlCourse.SelectedValue &
        "<br/>Date : " & calDate.SelectedDate.ToShortDateString()
    Else
        lblOutput.Text = "Please accept Terms & Conditions"
    End
```

Practical 4(b): Registration Form using Validation Controls

RegularExpressionValidator - ValidationExpression

\\w+([_+.’]\w+)@\w+([-.]\w+)\.\w+([-.]\w+)*

Register Button code

```
protected void btnSubmit_Click(object sender, EventArgs e)
{
    if (Page.IsValid) // Agar saare validation pass ho gaye
    {
        lblOutput.Text = "Registration Successful";
    }
}
```

web.config

```
<appSettings>
    <add key="ValidationSettings:UnobtrusiveValidationMode" value="None" />
</appSettings>
```

Practical 4(c): AdRotator Control

```
<Advertisements>
    <Ad>
        <ImageUrl>Images/ad1.jpg</ImageUrl>
        <NavigateUrl>https://www.google.com/</NavigateUrl>
        <AlternateText>Google Ad</AlternateText>
        <Impressions>50</Impressions>
    </Ad>

    <Ad>
        <ImageUrl>Images/ad2.jpg</ImageUrl>
        <NavigateUrl>https://www.microsoft.com/</NavigateUrl>
        <AlternateText>Microsoft Ad</AlternateText>
        <Impressions>50</Impressions>
    </Ad>
</Advertisements>
```

Practical 5(a): Different Types of Cookies

Code for Save Cookie Button

```
protected void btnSave_Click(object sender, EventArgs e)

{
    // Session Cookie - Browser band hone par delete ho jayega
    HttpCookie sessionCookie = new HttpCookie("SessionUser");
    sessionCookie.Value = txtName.Text;
    Response.Cookies.Add(sessionCookie);
    // Persistent Cookie - 7 din tak rahega
    HttpCookie persistentCookie = new HttpCookie("PersistentUser");
    persistentCookie.Value = txtName.Text;
    persistentCookie.Expires = DateTime.Now.AddDays(7);
    Response.Cookies.Add(persistentCookie);
    lblOutput.Text = "Cookies Saved Successfully";
    }
}
```

Code for Show Cookie Button

```
protected void btnShow_Click(object sender, EventArgs e)

{
    string sessionData = Request.Cookies["SessionUser"].Value;
    string persistentData = Request.Cookies["PersistentUser"].Value;
    lblOutput.Text = "Session Cookie: " + sessionData +
    "<br/>Persistent Cookie: " + persistentData;
}
```

Practical 5(b): Product Table and Insert Operation using Entity Framework Core

Products.cs

```
namespace ProductEFCore.Models
{
    public class Products
    {
        public int Id { get; set; }
        public required string ProductName { get; set; }
        public double Price { get; set; }
    }
}
```

AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using ProductEFCore.Models;
namespace ProductEFCore.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options)
        {
        }
        public DbSet<Products> Products { get; set; }
    }
}
```

appsettings.json

```
"AllowedHosts": "",
"ConnectionStrings": {
    "MyConnection": "Server=(localdb)\\
MSSQLLocalDB;Database=ProductDB;Trusted_Connection=True;TrustServerCertificate=True",
    "AppDbContext": "Server=(localdb)\\
mssqllocaldb;Database=AppDbContext;Trusted_Connection=True;MultipleActiveResultSets=true
"
}
```

Program.cs

```
using ProductEFCore.Data;
using Microsoft.EntityFrameworkCore;
Before:
var app = builder.Build();
Add:
builder.Services.AddDbContext<AppDbContext>(options =>
options.UseSqlServer(
    builder.Configuration.GetConnectionString("MyConnection"),
    sqlOptions => sqlOptions.EnableRetryOnFailure()
```

Package Manager Console

```
Add-Migration InitialCreate
Update-Database
```

ProductEFCore.csproj (paste before </Project> tag)

```
<ItemGroup>
    <NuGetAuditSuppress Include="https://github.com/advisories/GHSA-g4vj-cjjj-v7hg" />
</ItemGroup>
```

Practical 5(c): CRUD Operations on Employee Table using EF Core

Employee Model

```
{
    public class Employee
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public string Department { get; set; }
        public double Salary { get; set; }
    }
}
```

AppDbContext.cs

```
namespace EmployeeCRUD.Models
using EmployeeCRUD.Models;
using Microsoft.EntityFrameworkCore;
namespace EmployeeCRUD.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options)
        {
        }
        public DbSet<Employee> Employees { get; set; }
    }
}
```

appsettings.json

```
"ConnectionStrings": {
    "MyConnection": "Server=(localdb)\\
MSSQLLocalDB;Database=EmployeeDB;Trusted_Connection=True;TrustServerCertificate=True",
    "AppDbContext": "Server=(localdb)\\
mssqllocaldb;Database=AppDbContext;Trusted_Connection=True;MultipleActiveResultSets=true
"
}
```

Program.cs

```
using Microsoft.EntityFrameworkCore;
using EmployeeCRUD.Data;
Before:
Var app=builder.Bulid();
Add:
builder.Services.AddDbContext<AppDbContext>(options =>
options.UseSqlServer(
builder.Configuration.GetConnectionString("MyConnection")));
```

Package Manager Console

```
Add-Migration IntialCreate
Then:
Update-Database
```

Practical 5(d): Authentication and Authorization in ASP.NET Core

HomeController.cs

```
using Microsoft.AspNetCore.Authorization;
Add new Method
[Authorize]
public IActionResult Secret()
{
    return View();
}
```

Secret.cshtml

```
<h2>Authorized Users Only</h2>
```

Package Manager Console

```
Update-Database
```